

KV Cache Optimization for Efficient LLM Inference

Rishika Mamidibathula[†], Suhas Morisetty^{*}, Ziheng Wang^{*}, Tony Tian^{*}

^{*}Department of Computer Science, [†]Department of Data Science, Columbia University

COMS E6998: High Performance Machine Learning, Spring 2026

{rm4318, vm2825, zw3153, jt3640}@columbia.edu

Abstract—Autoregressive large-language-model (LLM) decoding is fundamentally bounded by key-value (KV) cache memory bandwidth. At batch size 32 and a 4K context, Llama-2-7B alone requires roughly 65 GB for the KV cache, nearly saturating an H100 80 GB GPU. We present a controlled benchmark of five KV optimization techniques drawn from four compression families—per-token quantization (KIVI 2/4-bit), dynamic sparse selection (TopK / TokenSelect), one-shot eviction (SnapKV), and latent projection (TransMLA)—together with a quantized-sparse hybrid (KIVI×TopK), all evaluated under a single unified harness that fixes the model, hardware, decode loop, prompts, and random seeds across runs. KIVI 4-bit preserves baseline LongBench quality with a $1.76\times$ decode-throughput gain at batch 32, while KIVI 2-bit trades -6.2% overall quality for a $1.93\times$ gain at batch 32 and lifts the operational batch ceiling from 32 to 128 (where baseline OOMs). SnapKV marginally improves overall quality at no per-step decode cost, though its one-shot eviction heuristic may not generalise uniformly beyond controlled benchmarks. TopK $K = 1024$ matches baseline quality and gains substantially on retrieval-heavy tasks. MLA yields a consistent $3.9\times$ KV reduction even at batch 1, but the available checkpoint regresses on output quality and requires retraining at longer context to be production-ready. The hybrid method stacks the memory savings of both compression axes, achieving a 50% reduction at 32K context; however, decode throughput degrades due to a non-optimal kernel implementation that does not yet fuse scoring, selection, and attention into a single pass—suggesting the latency gap is an engineering artifact rather than a fundamental limitation. Overall, each method occupies a distinct point in the quality–memory–latency space, and the right choice depends on whether batch size, context length, or output fidelity is the binding constraint.

Index Terms—KV cache, LLM inference, quantization, sparse attention, token eviction, multi-head latent attention, throughput, memory efficiency

I. INTRODUCTION

LLM inference proceeds in two phases. Prefill processes the prompt in a single forward pass; decode then emits one token per step. Each decode step rereads every cached key and value tensor from HBM, making decode strongly memory-bandwidth bound rather than compute-bound. The cost grows linearly with sequence length, batch size, and depth. For Llama-2-7B [2] (32 layers, 32 heads of dimension 128, FP16) at 4096 tokens and batch size 32, the cache occupies $2 \times 32 \times 32 \times 128 \times 4096 \times 32 \times 2 \approx 65$ GB [1]—leaving little room for activations on an 80 GB GPU and pushing larger configurations into out-of-memory territory.

Research on KV reduction has converged on four principal compression families. *Quantization* reduces bits per element while keeping all tokens [3], [4]. *Sparse selection* attends to a dynamically chosen subset of tokens at each decode step [5],

[6]. *Eviction* permanently discards low-importance tokens after prefill [8], [9], [10]. *Latent projection* stores a compressed low-rank representation instead of full K/V tensors [11], [12]. Cross-family comparison is difficult in practice because published evaluations rarely share the same model, hardware, or decode loop. This work addresses that gap with a unified harness whose only variable is the KV policy.

Our contributions are: (i) a single-codebase benchmark of five KV optimization techniques and a quantized-sparse hybrid on Llama-2-7B; (ii) empirical evidence that KIVI 4-bit and SnapKV 0.4 deliver lossless or marginally better LongBench quality; and (iii) full reproducibility via Modal cloud scripts and a public W&B dashboard at <https://wandb.ai/rm4318-columbia-university/kv-cache-hpml>.

II. LITERATURE REVIEW

KV Cache Compression (Quantization). Low-bit quantization reduces bytes per stored token while preserving full token coverage. KIVI [3] applies asymmetric quantization with per-channel scales for keys and per-token grouped quantization (group size 32) for values at 2–4 bit precision, retaining a recent-token window in FP16 for short-range fidelity; quantization is applied strictly at inference time without retraining or weight modification. At 2-bit it achieves roughly $8\times$ cache compression, at 4-bit roughly $4\times$. KVQuant [4] extends this with non-uniform codebooks and per-channel smoothing, reaching near-lossless compression at 1-bit at the cost of greater overhead.

KV Cache Low-Rank Approximation. Low-rank methods reduce per-token dimensionality rather than token count by projecting cached KV tensors into a lower-dimensional space. xKV [13] applies layer-group-wise SVD directly to cached KV tensors, stores the projected coefficients, and reconstructs approximate K/V at attention time—remaining strictly within inference scope by operating on the cache rather than model weights. DeepSeek-V2 [11] introduced Multi-Head Latent Attention (MLA), which stores a compressed latent vector c_t per token and expands to full K/V via learned up-projections, with Attention Absorption amortizing the projection cost. TransMLA [12] retrofits this latent structure onto an existing Llama-2 checkpoint, enabling evaluation of low-rank projection without full retraining.

KV Cache Retention Management (Eviction). Eviction methods permanently discard low-importance tokens to reduce stored token count while preserving high-utility context. H2O [8] identifies “heavy-hitter” tokens by cumulative attention

mass and retains a recent window alongside these high-importance tokens. SnapKV [9] performs a single one-shot eviction after prefill by aggregating attention weights from a small observation window and retaining the top-scored tokens per head, producing a fixed-size compressed cache with no per-step overhead. Ada-KV [10] extends this by allocating the eviction budget adaptively per head rather than uniformly, reflecting varying information density across attention heads.

Token-Level KV-Centric Scheduling (Sparse Selection).

Unlike eviction, token-level scheduling reduces effective attention work without permanently removing KV states—masked tokens remain stored and may be reactivated. At each decode step, a lightweight importance score (approximate $q \cdot k$) is computed and only the top- K tokens participate in attention, with the remainder masked. TokenSelect [5] implements this with a fused Triton scoring kernel and a cosine-similarity reuse cache to avoid re-scoring when consecutive queries are similar. RefreshKV [7] augments this with periodic full-context refresh to recover information from previously masked tokens. RocketKV [6] adds a two-stage compression pipeline to further reduce prefill and decode costs in long-context settings.

III. METHODOLOGY

A. Model and Hardware

All experiments use `meta-llama/Llama-2-7b-chat-hf` (32 layers, 32 heads, head dim 128, FP16 weights) on a single NVIDIA H100 80GB HBM3 GPU provided by Modal cloud compute. The runtime stack is pinned: PyTorch 2.4.1, HuggingFace Transformers 4.44.2, CUDA 12.1, Triton 3.0, flash-attn 2.5. The MLA experiments additionally use `botsoxc/llama2-7b-chat-mla-2048-8` [12] (2048-dim latent, 256-token calibration context, no further finetuning).

B. Datasets and Metrics

Quality. LongBench [14], six tasks $\times$ 20 examples = 120 examples per method: `qasper` (scientific QA, F1), `multifieldqa_en` (multi-document QA, F1), `triviaqa` (open-domain QA, F1), `2wikimqa` (multi-hop QA, F1), `multi_news` (summarization, ROUGE-L), `lcc` (code completion, edit similarity). Inputs are middle-truncated to 4096 tokens following the KIVI paper convention.

Throughput. Synthetic fixed-size batches with prefill 1024 and generation 512 tokens, swept over batch sizes 1–256. Five batches per configuration; the first is dropped as warmup; the mean of the remaining four is reported.

Long context. Prefill 32,768 tokens, generation 512, batch size 1, used for TopK-Flash and the KIVI $\times$ TopK hybrid.

C. Profiling Tools

Memory: `torch.cuda.max_memory_allocated()` reset before each batch; this measures peak, not steady-state. **Timing:** `torch.cuda.synchronize()` + `time.perf_counter()` brackets prefill (TTFT, time to first token) and the decode loop (throughput in tokens per second over steps 2 to G). Operator-level traces are captured

with `torch.profiler` and exported to Chrome trace. All runs are logged to Weights & Biases.

D. Method Implementations

All methods implement a shared `MethodWrapper` interface in `methods/` and run through an identical generate loop in `benchmark/runner.py`.

Baseline (FP16): dense attention with no KV modification.

KIVI [3]. Keys are quantized per-channel to 2 or 4 bits (grouped across the sequence dimension per head); values are quantized per-token with group size 32. The last 128 tokens are kept in FP16 (the residual buffer). Quantization fuses INT4 packing in a Triton kernel (`triton_quantize_and_pack`); GEMV dequantizes on-the-fly via `cuda_bmm_fA_qB_outer`. The cache is stored as a 9-tuple per layer (quantized blocks + scales + zeros for K and V plus the FP16 residual).

TopK / TokenSelect [5]. The full FP16 cache is retained; only the attention *scope* is sparse. Each decode step divides the cache into three non-overlapping regions: a fixed sink window (first 128 tokens), a dynamically selected top- K middle block, and a fixed local window (last 512 tokens). Scoring is per-layer-independent: a Triton kernel (`_topk_score_kernel`) fuses $QK^T/\sqrt{d}$, head-wise softmax (soft voting per paper Eq. 7), and a cross-head sum into a single GPU launch, then `torch.topk` selects the top- K middle indices. An optional max-pool smoothing (kernel size configurable) may be applied before top- K to cluster neighbouring tokens. A per-layer cosine-similarity cache (threshold 0.9) reuses the previous step’s indices when consecutive query fingerprints are sufficiently similar, avoiding redundant scoring. Because the selected subset is shorter than the original cache, a `true_seq_length` counter (= full accumulated cache length) is passed as `position_ids` at each decode step to keep RoPE indices correct. At long context (32K) a paged variant, **TopK-Flash**, uses a 256-token paged KV pool with `flash_attn_with_kvcache` for zero-copy selective attention, eliminating the ~ 52 GB contiguous allocation that standard flash-attn requires.

SnapKV [9]. A single one-shot eviction is performed immediately after prefill; decode then runs on the fixed compressed cache at zero per-step overhead. The algorithm has three stages. *Vote:* attention weights from the last $W = 32$ query positions (the *observation window*) are summed over the prefix per head, yielding an importance score of shape (batch, heads, prefix_len). Selection is done *per head*—averaging across heads would erase per-head specialisation. *Pool:* a 1D max-pool with kernel size 7 and same-padding is applied per head across the prefix axis *before* top- k selection to cluster neighbouring tokens and prevent fragmented picks (matching paper Section 4.3). *Snap:* the top- $\lfloor r \times S \rfloor$ prefix positions per head are gathered (budget ratio $r=0.4$, sequence length S); the full observation window is then concatenated back unconditionally, so no special sink bookkeeping is needed for it. Optionally, a configurable `sink_size` prepends the first few tokens (attention sinks) to the retained set. After

eviction the DynamicCache is shorter than the original prompt; a `true_seq_length` counter (= original prefill length + decode steps elapsed) is threaded to each decode step so that RoPE position indices remain accurate despite the shortened cache.

MLA / TransMLA [12]. Each Llama-2 attention layer is replaced with a DeepSeek-V3-style MLA module. A down-projection $\mathbf{W}_{kv}^A$ maps hidden states to a low-rank latent $c_t^{\text{raw}} \in \mathbb{R}^{d_c}$ and a RoPE component $k_t^R \in \mathbb{R}^{d_r}$ (split from a single linear), where $d_c = 2048$ (`kv_lora_rank`) and $d_r = 64$ (`qk_rope_head_dim`) for the evaluated checkpoint. An up-projection $\mathbf{W}_{kv}^B$ expands c_t back to full ($\mathbf{K}_{\text{nope}}, \mathbf{V}$) tensors at attention time.

Our implementation uses a custom `MLALatentCache` that stores *only* the normalised latent $\tilde{c}_t = \text{LayerNorm}(c_t^{\text{raw}})$ and the RoPE-applied rotation $\hat{k}_t^R$ per token per layer, rather than the fully expanded K/V pair. At each decode step, $\mathbf{W}_{kv}^B(\tilde{c}_{1:t})$ is evaluated over all accumulated latents to reconstruct K and V on-the-fly—an $O(t)$ compute cost that trades FLOP for memory. This avoids the pitfall of the naïve path (storing expanded K/V in a `DynamicCache`), which would be *larger* than the baseline GQA cache given the expanded head dimensions. Per-token cache footprint for the latent path: $(2048+64) \times 2 \text{ B/layer} \times 32 \text{ layers} \approx 132 \text{ KB/token}$, versus $\approx 512 \text{ KB/token}$ for the FP16 baseline ($32 \text{ heads} \times 128 d_k \times 2 \times 2 \text{ B} \times 32 \text{ layers}$), yielding the measured $3.9\times$ KV reduction (e.g., $1207 \rightarrow 311 \text{ MB}$ at prefill 2048, BS=1).

KIVI×TopK Hybrid. Prefill uses `F.scaled_dot_product_attention` (flash-attn backed, $O(T)$ memory) to avoid 32K OOM. Decode scores all tokens via `cuda_bmm_fA_qB_outer` over INT4-packed K, selects $n_{\text{sink}} + K + n_{\text{local}}$ positions, and gathers the corresponding INT4 V rows into a contiguous buffer fed directly into `cuda_bmm_fA_qB_outer`—no intermediate FP16 V is materialized.

IV. EXPERIMENTAL RESULTS

A. LongBench Quality

Table I reports per-task and overall LongBench scores. KIVI 4-bit and TopK $K=1024$ both achieve 0.292 (+0.001 absolute, +0.3% relative over baseline 0.291)—within evaluation noise for 20 examples per task but consistent across runs. SnapKV 0.4 reaches 0.295 (+0.004 absolute, +1.4% relative).

Per-task patterns reveal two opposite breakdown mechanisms when compression is too aggressive. KIVI 2-bit (0.273) and TopK $K=512$ (0.274) end up at essentially the same overall score, but for different reasons. KIVI’s quantization noise hurts qasper (−22%) and 2wikimqa (−40%), tasks that require precise recall of sparse mid-context facts. TopK $K=512$ hurts `multifieldqa_en` (−38%) because answer spans deeper than $n_{\text{sink}} + n_{\text{local}}$ are pruned. Conversely, TopK beats baseline on TriviaQA across all K (+13 pp to +20 pp absolute): the answer appears in multiple paraphrases, so sparse selection drops distracting copies and concentrates attention. SnapKV’s one-shot vote actively aligns selection with answer-bearing positions, boosting triviaqa (+4.3 pp) and `multifieldqa`

TABLE I
LONGBENCH QUALITY (F1 / ROUGE-L / EDIT-SIM), 6 TASKS $\times$ 20 EX.
TASKS: (1) QASPER, (2) MFQA, (3) TRIVIAQA, (4) 2WIKIMQA, (5) MNEWS, (6) LCC.

Method	(1)	(2)	(3)	(4)	(5)	(6)	Avg
Baseline (FP16)	.172	.380	.644	.083	.181	.287	.291
KIVI 4-bit	.194	.360	.643	.083	.188	.286	.292
KIVI 2-bit	.133	.382	.608	.050	.191	.277	.273
TopK $K=2048$	.145	.329	.731	.075	.178	.282	.290
TopK $K=1024$	.153	.281	.773	.089	.183	.273	.292
TopK $K=512$	.145	.237	.751	.050	.178	.281	.274
SnapKV 0.4	.153	.387	.687	.083	.181	.277	.295
Baseline (2K trunc) [‡]	.124	.298	.375	.089	.180	.362	.238
MLA (2K trunc) [‡]	.036	.101	.064	.043	.103	.140	.081

evaluation uses 2048-token middle truncation rather than 4096 to match the checkpoint’s calibration context; the matched 2K-truncation baseline (0.238) is the appropriate reference for the MLA row, yielding −66% relative. Numbers in this block are not directly comparable to the 4K-truncation rows above.

TABLE II
DECODE THROUGHPUT (TOK/S) AND PEAK GPU MEMORY VS. BATCH SIZE.
PREFILL 1024, GEN 512. “—” = OOM. SPEEDUP VS. FP16 BASELINE.

BS	Decode tok/s					Peak mem (GB)			
	Base	4b	Spd	2b	Spd	Base	4b	2b	MLA
1	70	29	0.41×	30	0.43×	15.1	14.3	14.0	13.8
2	135	58	0.43×	61	0.45×	16.7	14.8	14.4	—
4	242	109	0.45×	123	0.51×	20.0	16.0	15.3	—
8	351	216	0.62×	243	0.69×	26.4	18.4	17.1	—
16	438	404	0.92×	480	1.10×	39.3	23.7	20.7	—
32	497	873	1.76×	957	1.93×	65.0	33.6	27.8	—
64	—	1186	∞	1295	∞	—	53.6	42.1	—
128	—	—	—	1517	∞	—	—	70.7	—

Columns “4b” and “2b” are KIVI 4-bit and 2-bit respectively. KIVI 4-bit OOMs at BS=128 because its per-token footprint is twice that of 2-bit; KIVI 2-bit is the only configuration that fits BS=128 on a single H100.

(+0.7 pp) while only hurting qasper (−2.0 pp) where evidence spans fall outside the 32-token observation window. The MLA row, evaluated against its matched 2K-truncation baseline (0.238), drops to 0.081—a −66% relative regression—with the largest collapses on retrieval-heavy tasks (triviaqa −83%, `multifieldqa` −66%); see §V-C for the underlying cause.

B. Throughput and Memory

Table II shows the KIVI throughput sweep at prefill 1024, generation 512, for both 4-bit and 2-bit precision.

Crossover at BS=16. KIVI 4-bit decode throughput is $0.4\text{--}0.6\times$ baseline at BS 1–8, reaches near-parity at BS=16 (404 vs. 438 tok/s, $0.92\times$), and crosses to $1.76\times$ at BS=32; KIVI 2-bit follows the same shape but crosses earlier (at BS=16, $1.10\times$) and reaches $1.93\times$ at BS=32. The mechanism is straightforward: at small batch the decode $\mathbf{QK}^\top$ is a GEMV, and INT4 GEMV does not map to H100 tensor-core units—it is roughly $2.5\times$ slower than FP16 GEMV. At large batch the same operation becomes a near-GEMM where the HBM bandwidth saving (the 2-bit cache is $2.3\times$ smaller than baseline at BS=32: 27.8 vs. 65.0 GB; 4-bit is $1.9\times$ smaller at 33.6 GB) outweighs dequantization cost.

TABLE III
LONG-CONTEXT RESULTS. PREFILL 32,768 TOKENS, GEN 512, BS=1.

Method	TTFT (ms)	Spdup	Peak mem	Saving
Baseline (FP16)	2627	1.00×	52.1 GB	—
TopK-Flash	1761	1.49×	42.1 GB	−19% Baseline
SnapKV 0.4	—	—	26.0 GB [†]	−50%
KIVI×TopK 4b	—	0.61×	25.8 GB	−50%

OOMs at 64K; TopK-Flash succeeds at 65.8 GB. [†]SnapKV peak memory at 32K is interpolated from short-context measurements; 32K SnapKV throughput not directly benchmarked.

Capacity extension. Baseline OOMs at BS=64; KIVI 4-bit reaches BS=64 (53.6 GB, 1186 tok/s) and KIVI 2-bit reaches BS=128 (70.7 GB, 1517 tok/s)—about 3× the maximum throughput ever achievable with the FP16 baseline on this GPU. This is the strongest practical argument for KIVI: it raises the per-GPU concurrency ceiling, not just speed at fixed batch. KIVI 4-bit is the quality-preserving operating point (0.292 vs. baseline 0.291); KIVI 2-bit pays −6.2% overall quality (0.273) for an additional batch doubling and modestly higher throughput.

TTFT overhead. KIVI’s TTFT is 18–66% worse across batch sizes (998 vs. 1659 ms at BS=32 for 2-bit; 4-bit is essentially identical) because the full KV cache is quantized and packed at the end of prefill—a cost proportional to $B \times S$. For latency-sensitive interactive workloads this is a real penalty; for throughput-oriented offline inference it is amortized over hundreds of decode steps.

MLA at B=1. At gen=256, MLA achieves 43–44 tok/s vs. 34 tok/s baseline (1.26–1.32×) across prefill lengths 512–2048. Even single-request decode is partially HBM-bandwidth bound: the 3.9× KV reduction (e.g., 1207 → 311 MB at $p=2048$) reduces per-step HBM traffic proportionally. The speedup narrows at gen=512 because the MLA up-projection cost grows with output length.

C. Long-Context Results (32K)

TopK-Flash. At 32K context the 256-token paged KV pool eliminates the 52 GB contiguous allocation that standard flash-attn requires, reducing TTFT by 33% and peak memory by 19%. The win is not from sparsity but from allocation and memory-access-pattern improvement. Baseline OOMs at 64K; TopK-Flash succeeds at 65.8 GB.

KIVI×TopK hybrid. Combining INT4 storage with sparse attention cuts peak memory 50% (52.1→25.8 GB) but decode throughput drops 39% (21.0→12.9 tok/s). The slowdown has two attributable sources: (a) QK^T must still score *all* 32K tokens every step before top- K can be applied—the scoring cost is identical to dense baseline, plus the overhead of topk + sort + gather on top; (b) at batch size 1 and 32K context the INT4 V GEMV inherits the same dequantization penalty as standalone KIVI, where tensor-core utilisation is low and the dequantization overhead is not amortised. LongBench quality remains at 0.297, within noise of 0.291, confirming the compression is semantically lossless.

TABLE IV
DEPLOYMENT FIT SUMMARY.

Method	KV ratio	Quality	Best regime
KIVI 4-bit	4×	Lossless (.292)	Batch serving $\geq$ BS=16
KIVI 2-bit	8×	−6.2% (.273)	Max-batch, accept noise
TopK $K=1024$	1× [†]	Lossless (.292)	Single-req, long context
SnapKV 0.4	2.5×	+1.4% (.295)	Any; quality-neutral
MLA (B=1)	3.9×	−66%* needs retrain	After finetuning at $\geq 4K$

[†]TopK keeps full FP16 cache; only attention scope is sparse. *Measured against matched 2K-truncation baseline (0.238).

Importantly, we believe the throughput regression is *not* fundamental to the hybrid strategy but rather an artifact of the current kernel implementation. The scoring kernel reads the full INT4 K cache sequentially before any selection, and the subsequent gather plus INT4 V attention are not fused—each is a separate kernel launch with its own HBM round-trip. A tighter implementation that fuses scoring, index selection, and INT4 V attention into a single paged kernel (analogous to what FlashInfer or FlashAttention-3 do for dense attention) could in principle recover the latency by eliminating redundant memory traffic. Similarly, a hierarchical or lazy scoring scheme—e.g., scoring only a coarse page-level summary first and expanding to token-level only for shortlisted pages—would reduce the $O(S)$ scoring cost toward $O(K)$. We therefore view the hybrid as a promising direction that merits further kernel-level engineering: the memory savings from orthogonal INT4 and sparse-selection compression are real and additive, and achieving the best of both worlds (throughput *and* memory) appears within reach with a purpose-built fused kernel.

V. DISCUSSION

A. Deployment Recommendations

Table IV summarizes the operating point of each method.

KIVI 4-bit is the best practical default for high-throughput batch inference: it costs nothing in quality, raises max batch from 32 to 64 on a single H100 (and 128 if 2-bit is acceptable), and 1.76×es throughput at BS=32. The cost is TTFT latency (+18–66%), which matters for interactive applications. KIVI 2-bit is the right choice when maximum concurrency dominates and a −6.2% quality cost is tolerable.

SnapKV 0.4 is the right choice when quality must be preserved and per-step overhead must be zero. Its 2.5× KV reduction has no decode-time cost because the compressed cache is built once after prefill. That said, our benchmark uses only 20 LongBench examples per task with inputs truncated to 4096 tokens, and the one-shot eviction heuristic (voting over a 32-token observation window) may not generalise equally well to all real-world distributions. Tasks where critical evidence is spread uniformly across the prompt, or where the most relevant tokens do not align with the final observation window, could see larger quality degradation than our controlled evaluation suggests. More deliberation is needed—across longer contexts, more diverse task types, and multi-turn settings—before SnapKV can be recommended as a broadly quality-neutral drop-in.

TopK $K = 1024$ is suited to single-request, long-context use ($> 32K$) where the paged variant (TopK-Flash) provides real TTFT and memory gains. At short context and low batch, scoring overhead dominates the saved attention work.

MLA shows the theoretical ceiling of latent projection: $3.9\times$ KV reduction and $1.3\times$ decode speedup are repeatable benefits even at $B=1$, but production-quality output requires a checkpoint trained at longer context with higher latent rank.

B. Kernel-Level Observations

INT4 GEMV inefficiency. `cuda_bmm_fA_qB_outer` performs on-the-fly INT4 $\rightarrow$ FP16 dequantization per GEMV. H100 tensor cores target INT8/FP16 GEMM; INT4 GEMV falls back to scalar or vector-INT4 paths. Bandwidth saving only exceeds this overhead when decode approaches GEMM at $BS \geq 16$.

KIVI residual buffer hot path. At short generation lengths ($< residual_length \times n_{layers}$), nearly all attention concentrates in the FP16 residual; KIVI provides minimal storage benefit in this regime. This is also why KIVI TTFT is roughly $3\times$ slower: the full prefill quantization is paid upfront.

Triton fused scoring. `_topk_score_kernel` fuses $QK^T + \text{softmax} + \text{head-sum}$ into a single launch, reducing Python-dispatch overhead $2\text{--}3\times$ vs. sequential PyTorch ops. At 1K–4K sequence lengths the critical path remains HBM bandwidth, not compute or launch latency.

Hybrid stays in INT4. The hybrid keeps data in INT4 across the gather $\rightarrow$ attention boundary by passing the gathered packed tensor directly into `cuda_bmm_fA_qB_outer`, avoiding an FP16 V materialization that would have cost an extra allocation and HBM round-trip per layer per step.

C. Limitations

Single architecture. Results are specific to Llama-2-7B with multi-head attention (32 heads, 32 KV heads). Models with grouped-query attention (Llama-3, Mistral) have fewer KV heads, which changes the compression profile.

Small evaluation set. LongBench results use 20 examples per task. Some per-task deltas (e.g., qasper KIVI 2-bit -22%) may partially reflect small- n variance, although the direction is consistent across tasks.

MLA checkpoint and truncation. The available TransMLA checkpoint was calibrated at 256-token context with rank-8 latent compression and without task finetuning—insufficient capacity for 4K-token retrieval-heavy LongBench tasks. We accordingly evaluated MLA against a matched 2K-truncation baseline (0.238) rather than the 4K-truncation baseline used for the other methods (0.291); the -66% quality gap reported in Table IV reflects this matched comparison. A properly trained checkpoint at higher rank (16–32) and longer context (4–8K) would likely recover most of this gap, as the underlying latent-projection mechanism produces a clean $3.9\times$ KV reduction and $1.3\times$ decode speedup.

Hybrid scoring cost. The 50% memory win comes with a 39% decode penalty because scoring must precede selection. Hierarchical or lazy-scoring approaches could decouple them.

VI. CONCLUSION

We conducted a controlled benchmark of five KV-cache optimization methods spanning four compression families on Llama-2-7B. KIVI 4-bit provides lossless quality with $4\times$ KV compression and $1.76\times$ throughput at $BS=32$ —our recommended default for batch-oriented serving when quality must be preserved; KIVI 2-bit extends the operational batch ceiling from 32 to 128 at a -6.2% quality cost. SnapKV marginally improves LongBench overall score with zero runtime overhead. TopK $K = 1024$ matches baseline quality and gains substantially on retrieval-heavy tasks. MLA delivers a consistent $3.9\times$ KV reduction and $1.3\times$ decode speedup at $B=1$ but requires a properly trained checkpoint for production quality. The KIVI $\times$ TopK hybrid stacks memory savings of both axes but pays a decode penalty until scoring cost can be reduced. No single method dominates all axes; practitioners should select based on whether batch size, context length, or output fidelity is the binding constraint.

Future work should address: (i) batched decode for TopK to amortize the per-step scoring cost and produce a KIVI-style crossover at $BS \geq 8\text{--}16$; (ii) MLA retraining at 4K–8K context with rank 16–32 to preserve $3.9\times$ compression while recovering QA quality; (iii) hierarchical or lazy scoring in the KIVI $\times$ TopK hybrid to decouple the memory win from the decode penalty; and (iv) cross-method composition, e.g., SnapKV eviction followed by KIVI quantization on the retained tokens to stack savings from orthogonal axes.

APPENDIX

AI USE DISCLOSURE

Per the HPML AI Use Policy.

Tools used: Claude (Anthropic) and GitHub Copilot.

Specific purpose: Debugging CUDA OOM errors during KIVI kernel integration; clarifying Triton kernel semantics (block-pointer arithmetic, mask construction, autotune configuration); polishing prose in sections drafted by team members.

Sections affected: `methods/kivi_kernels/` integration debugging, `methods/topk_selection.py` kernel review; §I–§VI prose polish on text first written by team members. No analytical reasoning, performance interpretation, or experimental result was generated by an AI tool.

How we verified correctness: All experimental numbers in this report were produced by running our own Modal scripts on H100 containers and logged to W&B; profiler-trace interpretations were cross-checked against the raw `torch.profiler` traces in `results/`; AI-suggested code was rewritten in our own words and re-validated to produce identical outputs to a hand-written reference before being committed.

By submitting this project, the team confirms that the analysis, interpretations, and conclusions are our own, and that any AI assistance is fully disclosed above. The same disclosure block appears in the project README.

REFERENCES

- [1] J. Jiang, P. Yang, R. Zhang, and F. Liu, “Towards efficient large language model serving: A survey on system-aware KV cache optimization,” *TechRxiv*, 2026.
- [2] H. Touvron *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv:2307.09288*, 2023.
- [3] Z. Liu *et al.*, “KIVI: A tuning-free asymmetric 2-bit quantization for KV cache,” in *Proc. ICML*, 2024.
- [4] C. Hooper *et al.*, “KVQuant: Towards 10 million context length LLM inference with KV cache quantization,” in *NeurIPS*, 2024.
- [5] W. Wu *et al.*, “TokenSelect: Efficient long-context inference via dynamic token-level KV cache selection,” in *Proc. EMNLP*, 2025.
- [6] P. Behnam *et al.*, “RocketKV: Accelerating long-context LLM inference via two-stage KV cache compression,” in *Proc. ICML*, 2025.
- [7] F. Xu, T. Goyal, and E. Choi, “RefreshKV: Updating small KV cache during long-form generation,” in *Proc. ACL*, 2025.
- [8] Z. Zhang *et al.*, “H2O: Heavy-hitter oracle for efficient generative inference of large language models,” in *NeurIPS*, 2023.
- [9] Y. Li *et al.*, “SnapKV: LLM knows what you are looking for before generation,” in *NeurIPS*, 2024.
- [10] Y. Feng *et al.*, “Ada-KV: Optimizing KV cache eviction by adaptive budget allocation,” in *NeurIPS*, 2025.
- [11] DeepSeek-AI, “DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model,” *arXiv:2405.04434*, 2024.
- [12] Y. He *et al.*, “TransMLA: Multi-head latent attention is all you need,” *arXiv:2502.07864*, 2025.
- [13] C.-C. Chang *et al.*, “xKV: Cross-layer SVD for KV-cache compression,” *arXiv:2503.18893*, 2025.
- [14] Y. Bai *et al.*, “LongBench: A bilingual, multitask benchmark for long context understanding,” *arXiv:2308.14508*, 2023.